StreetCare AI — data architecture: data flow and lineage

Where every dataset comes from, how it is cleaned, what it becomes and which keys join it — row counts, scores and sizes are read from the project's own files

Built from config/config.py · data_pipeline/ · modeling/ · knowledge/ · persistence/models.py

1 Sources · data/

Historical extracts — training & held-out evaluation

five CSVs as delivered: mixed units, three reference notations, stray spellings, duplicates

file	rows	cols	target	used for
report_history_train.csv	32,738	31	reported_again	repeat likelihood
report_history_eval.csv	8,202	31	reported_again	held out
repair_logs_train.csv	26,095	27	actual_crew_hours	expected crew hours
repair_logs_eval.csv	6,476	27	actual_crew_hours	held out
statement_sentences.csv	36,661	8	sentence_type	sentence type (9 classes)

Live inputs — data/input/

the open queue the service works on

file	rows × cols	after cleaning
open_defect_reports.csv	45 × 27	40 open
report_documents.csv	62 × 6	60 docs
street_works_register.csv	29 × 6	23 live

40 reports = latest revision of each reference
28 permits, 23 live · documents keyed by document_id
no outcome columns: nothing to leak

Policy documents — data/docs/

16 PDFs, numbered sections

- 5 Defect-Standard-*.pdf
- 8 Guide-*.pdf (photo, measure, …)
- 1 Highways-Policy.pdf
- 1 Report-Rules.pdf
- 1 Past-Report-Decisions-Casebook.pdf

casebook is indexed but never quoted in chat

2 Preparation · offline

run_data_pipeline.py · data_pipeline/

1. load every column as text · NULL_TOKENS → missing
2. drop exact duplicates and rows missing the target
3. SC-52158 · sc52158 · SC0052158 → one reference
4. fold category spellings to one canonical value
5. cm · inch · ft · min → mm · m · h (Decimal, half-up)
6. below minimum / above maximum → missing
7. keep the latest revision of each report
8. text: markup, entities, whitespace, CAPS → plain

rows received → kept

history train	32,738 → 30,569 (–2,169)
history eval	8,202 → 7,648 (–554)
logs train	26,095 → 25,537 (–558)
logs eval	6,476 → 6,337 (–139)
sentences	36,661 → 35,717 (–944)

dark = kept · light = removed

Live cleaning — at service start

tools/report_data_tools.py

the same cleaners as training:
`report_cleaning · document_cleaning`
`register_cleaning`

then, per open report, in plain code:
screening under the parsed standard
street-works count (live / conflicting)
sentence evidence + disagreements
repeat likelihood + expected crew hours

run_knowledge_indexing.py

knowledge/

`pdf_extraction`
text of each of the 16 PDFs
`chunk_splitting`
numbered sections → passages of
700 chars, 120 carried over;
a requirement is never split
`vector_index`
embed, persist; read-back check
of every section

3 Derived assets

processed/

5 clean CSVs · same columns

report_history_*_clean
repair_logs_*_clean
statement_sentences_clean

`output/`
`data_quality_report.json`
gaps, units as written, non-
canonical values per column

run_model_training.py · feature_builder.py · modeling/

fit on train only · scored on the held-out eval file · saved only if all three pass

Repeat likelihood
LogisticRegression
reported_again · 27 features
30,569 train / 7,648 eval
ROC AUC 0.835
minimum 0.80 (red) · scale 0.5–1
Brier 0.117

Expected crew hours
RandomForest · 300 trees
actual_crew_hours · 20 features
25,537 train / 6,337 eval
R² 0.902
minimum 0.86 (red) · scale 0.5–1
MAE 0.42 h

Sentence type
TF-IDF + LogisticRegression
sentence_type · 9 classes
28,573 train / 7,144 eval
macro-F1 0.978
minimum 0.95 (red) · scale 0.5–1
accuracy 0.977

report_queue

in memory
40 rows, built
once at import

ranked by code,
never by a model

+ details per
report

database/chroma_store/

ChromaDB · chroma.sqlite3
collection street_care_documents
205 passages · all-MiniLM-L6-v2
cosine distance
metadata: document_name,
section_number, passage_number,
carried_characters

facts + model figures copied into the review at the gate

4 Operational stores · online

database/streetcare.db

SQLite via SQLAlchemy · persistence/repository.py is the only module that queries it · keys are joined in code (no FOREIGN KEY constraints)

user_accounts — 7 cols · 16 seeded

PK	username	
	display_name · role	
	district	*inspectors only*
	password_hash	*PBKDF2-SHA256 ×200k*
	password_salt	*16 bytes*
UQ	session_token	*24 bytes · NULL = out*

1 : N username

chat_turns — 12 cols · every reply

PK	turn_id	
FK	username · user_role	→ username
	question_text · question_type	
	answer_text · source_documents	refusals too
	closest_distance · was_refused	

1 : N

agent_runs — 7 cols · one per review run

PK	run_id	*RUN-<12 hex>*
UQ	thread_id	*= checkpoint thread*
FK	coordinator_id	→ username
	report_reference	
	coordinator_request	
	status	*RUNNING → AWAITING → COMPLETE*
	llm_call_count	*5, then 7 / 6 / 5*

1 : 1 thread_id

checkpoint.db — LangGraph SqliteSaver

	checkpoints · writes	*2 tables*
FK	thread_id	→ agent_runs
	ReportReviewState	*paused at the gate*
	resume →	*order · update · record*

1 : 1

report_reviews — 37 cols · one per revision

PK	review_id	*REV-<12 hex>*
UQ	run_id	→ agent_runs
	report_reference · revision_number	
	defect_type · street_name …	*identity*
	requirements · met/not met/not doc	*screening*
	live / conflicting_permits	*register*
	coordination_triggers · assessment	
	evidence · disagreements	*JSON*
	repeat_likelihood · band	*model*
	expected / planned_crew_hours	*model*
	5 agent texts · recommended_decision	*LLM*
	status	*AWAITING → decided*
	order_document_text · details	*LLM 6*
	resident_update_text	*LLM 7*
	coordination_brief	*code (REFER)*

1 : 0..1

review_decisions — 8 cols · 0..1 per review

PK	decision_id	*autoincrement*
UQ	review_id	→ report_reviews
FK	run_id	→ agent_runs
	decision	*ORDER / INSPECT / REFER*
FK	decided_by · _role · _district	→ username
	decision_reason	*≥ 10 chars*

decided_by → user_accounts

1 : 5–7

llm_calls — 8 cols · 5–7 per run

PK	call_id	*autoincrement*
FK	run_id	→ agent_runs
	agent_name · model_name	
	prompt / response / total_tokens	
	latency_milliseconds	*accepted replies only*

Keys that join the data

`report_reference`
every CSV, documents, sentences,
runs, reviews, chat_turns · SC0052158

`revision_number`
latest kept; one review per
report revision

`street_name`
reports ↔ street_works_register ↔
district (INSPECTION_DISTRICT_STREETS)

`document_id`
report_documents ↔ evidence
findings ↔ disagreements

`run_id · thread_id`
agent_runs ↔ report_reviews ↔
llm_calls ↔ checkpoint.db

`review_id`
report_reviews ↔ review_decisions

`username`
user_accounts ↔ coordinator_id,
decided_by, chat_turns

Data rules enforced in code

Exact decimals
every unit conversion uses Decimal, ROUND_HALF_UP

No leakage
fit on train only; eval held out; train-vs-eval gap kept

Same cleaning
live open reports pass the training cleaners

Facts before prose
agents see tool-computed facts; they compute nothing

Need-to-know
advisers never see a likelihood or crew hours

Small groups
a street figure needs ≥ 5 reports behind it

Full audit
every LLM call and every chat turn is stored

Credentials
salted PBKDF2 hashes; tokens cleared on sign-out

One review
per report revision; a decision only once

Footprint on disk now

megabytes, from du on the Project folder

data/	17 MB
processed/	17 MB
output/models/	13 MB
chroma_store/	2.4 MB
checkpoint.db (+WAL)	0.74 MB
streetcare.db	0.13 MB

the live database stays small: text and JSON only

Read top to bottom: raw files are cleaned once offline into processed files, models and an index; the running service cleans the live queue the same way and writes every review, decision, LLM call and chat turn to SQLite.

PK primary key · UQ unique · FK logical reference joined in code. Counts: received → kept from output/data_quality_report.json; scores from output/model_metrics.json.

Raw / stored data · Deterministic code · Trained ML model · LLM-written data · Review & decision records · Access data · data flow · lineage into a record